

claude-windows / 42211f80-2169-4025-ad9d-99c1aa29ff70

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\42211f80-2169-4025-ad9d-99c1aa29ff70\subagents\workflows\wf_d4359b2a-456\agent-ad51c075f170c4f2c.jsonl`
- SHA-256: `8223cc315378fe41bf3413093d87693c77506c071f30c480422c7e4e125483a9`
- Source modified: `2026-06-06T15:22:57+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_d4359b2a-456`
- session_id: `42211f80-2169-4025-ad9d-99c1aa29ff70`
```

Transcript

1. user (2026-06-06T15:20:37.518Z)

You are mapping ONE subsystem of the "badminton-highlight-indexer" Python repo (cwd is the repo root) for a dense CODE MAP that a future AI engineer will read to ramp up FAST on the codebase without crawling files.

Subsystem: Segmenters (indexer)

Files: backend/indexer/base.py, backend/indexer/__init__.py, backend/indexer/wasb_hybrid.py, backend/indexer/tracknet_hybrid.py, backend/indexer/yolo_hybrid.py, backend/indexer/motion.py, backend/indexer/gemini.py

READ each listed file (use Read; use Grep/Glob to resolve any directory globs to actual files). Then return, per file:

- path (exact, repo-relative, forward slashes)
- role: one terse line ? what the file is for
- symbols: the IMPORTANT classes / functions / dataclasses / module-level constants & registries. For each: name, kind (class|func|dataclass|const|registry), a terse signature (params + return where useful), and a one-line role. SKIP trivial private helpers and boilerplate.
- contracts: exact data shapes this file defines or consumes (dict keys, CSV column orders, tuple layouts, dataclass fields, return types). Be precise ? these are the cross-module interfaces.
- edges: calls/imports to OTHER files that matter for dataflow, each as "thisSymbol -> otherFile.thing (why)".
- gotchas: non-obvious constraints, footguns, platform/WSL/GPU/threadpool specifics, dead paths, defaults that bite.

Rules: be terse and ACCURATE. Every symbol and path MUST actually exist in the file ? do NOT invent or guess. If something is notable but you're unsure, omit it. Return via the structured schema.

2. assistant (2026-06-06T15:20:43.837Z)

I'll map the Segmenters subsystem. Let me read all the listed files.

3. user (2026-06-06T15:20:44.858Z)

```
1      from abc import ABC, abstractmethod
2      from typing import List, Dict, Any, Type, Optional
3      from backend.database import Database
4
5      class BasePlaySegmenter(ABC):
6          def __init__(self, db: Database, config: Dict[str, Any]):
7              self.db = db
8              self.config = config
9
```

```

10     @property
11     @abstractmethod
12     def name(self) -> str:
13         """Unique identifier for this segmenter."""
14         pass
15
16     @property
17     @abstractmethod
18     def description(self) -> str:
19         """Human-readable explanation of this segmenter."""
20         pass
21
22     @abstractmethod
23     def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
24         """
25         Processes a video to detect play segments, populates SQLite database,
26         and returns list of segment representations.
27         """
28         pass
29
30     class SegmenterRegistry:
31         def __init__(self):
32             self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
33
34         def register(self, segmenter_cls: Type[BasePlaySegmenter]):
35             """Registers a new segmenter class under its name property."""
36             try:
37                 temp_inst = segmenter_cls(None, {})
38                 name = temp_inst.name
39             except Exception as e:
40                 raise ValueError(f"Failed to resolve name property from segmenter class
{segmenter_cls.__name__}: {e}")
41             self._segmenters[name] = segmenter_cls
42
43         def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
44             return self._segmenters.get(name)
45
46         def list_available(self) -> Dict[str, str]:
47             """Returns a dict of registered segmenter names and their descriptions."""
48             res = {}
49             for name, cls in self._segmenters.items():
50                 try:
51                     temp_inst = cls(None, {})
52                     desc = temp_inst.description
53                 except Exception:
54                     desc = "No description available."
55                 res[name] = desc
56             return res
57
58     segmenter_registry = SegmenterRegistry()
59

```

4. user (2026-06-06T15:20:45.406Z)

```

1     from backend.indexer.base import segmenter_registry, BasePlaySegmenter
2     from backend.indexer.motion import MotionPlaySegmenter
3     from backend.indexer.gemini import GeminiPlaySegmenter
4     from backend.indexer.yolo_hybrid import HybridYoloSegmenter
5     from backend.indexer.tracknet_hybrid import TrackNetHybridSegmenter
6     from backend.indexer.wasb_hybrid import WasbHybridSegmenter
7
8     __all__ = [
9         "segmenter_registry",
10        "BasePlaySegmenter",

```

```

11         "MotionPlaySegmenter",
12         "GeminiPlaySegmenter",
13         "HybridYoloSegmenter",
14         "TrackNetHybridSegmenter",
15         "WasbHybridSegmenter",
16     ]
17

```

5. user (2026-06-06T15:20:45.794Z)

```

1     """WASB-based hybrid play segmenter.
2
3     Identical three-phase shape to TrackNetHybridSegmenter, but Phase 2 uses **WASB**
4     (nttcom/WASB-SBDT, MIT) shuttle-trajectory detection via WSL2
5     (backend/detectors/wasb_runner.py) instead of TrackNetV4. WASB ships pretrained
6     badminton weights and runs today, so this is the live shuttle-motion substrate.
7     Phase 3 (AI semantic boundaries) and Phase 4 (DB) are shared with the other hybrids.
8
9     For the per-video *calibration* goal we mainly use Phase 2 (trajectory -> candidate
10    windows); the windowing thresholds (velocity_thresh / merge_gap / min_window_duration)
11    are what get tuned against golden rally labels.
12    """
13    import os
14    import time
15    import logging
16    import concurrent.futures
17    from typing import List, Dict, Any
18
19    import cv2
20
21    from backend.indexer.base import BasePlaySegmenter, segmenter_registry
22    from backend.utils.video import get_video_duration, extract_video_chunk
23    from backend.sports import sport_registry
24    from backend.providers import provider_registry
25    from backend.detectors.wasb_runner import WasbRunner, WasbConfig
26
27    logger = logging.getLogger(__name__)
28
29
30    def _fmt_ts(seconds: float) -> str:
31        seconds = max(0, int(seconds))
32        h, rem = divmod(seconds, 3600)
33        m, s = divmod(rem, 60)
34        return f"{h:02d}:{m:02d}:{s:02d}"
35
36
37    class WasbHybridSegmenter(BasePlaySegmenter):
38        @property
39        def name(self) -> str:
40            return "wasb_hybrid"
41
42        @property
43        def description(self) -> str:
44            return ("Two-stage pipeline: WASB shuttle-trajectory detection (WSL2, pretrained
45            badminton "
46                    "weights) for precise candidate rallies + AI provider for semantic
47            boundaries.")
48
49        @staticmethod
50        def _probe_fps_width(video_path: str) -> tuple:
51            cap = cv2.VideoCapture(video_path)
52            fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
53            width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
54            cap.release()
55            return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)

```

```

54
55     def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None,
56                         max_frames: int = None) -> List[Dict[str, Any]]:
57         sport_name = self.config.get("sport", "badminton")
58         try:
59             sport_profile = sport_registry.get(sport_name)
60         except KeyError as e:
61             logger.error(str(e))
62             return []
63
64         idx_cfg = self.config.get("indexing", {})
65         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
66         model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
67
68         api_key_env_var = f"{provider_name.upper()}_API_KEY"
69         api_key = os.environ.get(api_key_env_var)
70         if not api_key:
71             logger.error(f"{api_key_env_var} not set; cannot run the {provider_name}
handoff.")
72             return []
73         try:
74             provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
75         except KeyError as e:
76             logger.error(str(e))
77             return []
78
79         video_duration = get_video_duration(video_path)
80         if video_duration <= 0:
81             logger.error("Invalid video duration.")
82             return []
83         fps, frame_width = self._probe_fps_width(video_path)
84
85         wasb_cfg = WasbConfig.from_indexing_cfg(idx_cfg)
86         runner = WasbRunner(wasb_cfg)
87
88         w = idx_cfg.get("wasb", {})
89         velocity_thresh = w.get("velocity_threshold", 0.02)
90         merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
91         min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
92         handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
93         ai_max_workers = idx_cfg.get("ai_max_workers", 5)
94         log_candidates = idx_cfg.get("log_yolo_candidates", True)
95         store_candidates = idx_cfg.get("store_yolo_candidates", True)
96
97         detection_params = {
98             "detector": "wasb_hybrid",
99             "velocity_thresh": velocity_thresh,
100             "merge_gap": merge_gap,
101             "min_action_window_duration": min_window_duration,
102             "weights_path": wasb_cfg.weights_path,
103         }
104
105         ok, msg = runner.healthcheck()
106         if not ok:
107             logger.error("WASB WSL environment not ready: %s", msg)
108             return []
109         logger.info("WASB WSL env OK: %s", msg)
110
111         # --- Phase 2: shuttle trajectory -> candidate rally windows ---
112         t_start = time.time()
113         out_dir = os.path.join("output", "wasb")

```

```

114         csv_path = runner.run_predict(video_path, out_dir)
115         if not csv_path:
116             logger.error("WASB produced no trajectory; aborting.")
117             return []
118
119         points = runner.parse_trajectory_csv(csv_path)
120         logger.info("Parsed %d trajectory points (%d visible).",
121                     len(points), sum(1 for p in points if p.visible))
122
123         all_candidate_windows = runner.trajectory_to_action_windows(
124             points, fps=fps, frame_width=frame_width, chunk_start=0.0,
125             velocity_thresh=velocity_thresh, merge_gap=merge_gap,
126             min_window_duration=min_window_duration, chunk_index=0,
127         )
128         logger.info("Phase 2 (WASB) completed in %.1fs. Candidate windows: %d",
129                     time.time() - t_start, len(all_candidate_windows))
130
131         if log_candidates:
132             for cw in all_candidate_windows:
133                 logger.info(f"[WASB rally] {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
134                             f"({cw['end'] - cw['start']:.1f}s) | "
135                             f"peak_v={cw['peak_velocity']:.3f} "
136                             f"mean_v={cw['mean_velocity']:.3f}")
137
138         candidate_ids = [None] * len(all_candidate_windows)
139         if store_candidates:
140             for i, cw in enumerate(all_candidate_windows):
141                 stats = {
142                     "peak_velocity": cw["peak_velocity"], "mean_velocity":
143                     cw["mean_velocity"],
144                     "active_frame_count": cw["active_frame_count"], "track_id_count":
145                     cw["track_id_count"],
146                 }
147                 candidate_ids[i] = self.db.add_candidate_segment(
148                     video_id, detector="wasb_hybrid", start_time=cw["start"],
149                     end_time=cw["end"],
150                     chunk_index=cw["chunk_index"], confidence=min(1.0,
151                     cw["peak_velocity"]),
152                     stats=stats, detection_params=detection_params,
153                 )
154
155         if not all_candidate_windows:
156             return []
157
158         # --- Phase 3: AI provider handoff (shared pattern) ---
159         handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
160         os.makedirs(handoff_dir, exist_ok=True)
161         logger.info("Phase 3: %s validation on %d candidate windows...",
162                     provider_name, len(all_candidate_windows))
163
164         def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
165             w_start = max(0, window["start"] - handoff_padding)
166             w_end = min(video_duration, window["end"] + handoff_padding)
167             w_dur = w_end - w_start
168             w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
169             if not extract_video_chunk(video_path, w_start, w_dur, w_path,
170                                     reencode=True):
171                 return []
172
173             prompt = sport_profile.get_prompt(chunk_duration=w_dur)
174             segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
175                                                 label=f"Handoff {wi+1}")
176
177             valid = []
178             for seg in segments:
179                 try:

```

```

172         seg["start_time"] = float(seg["start_time"]) + w_start
173         seg["end_time"] = float(seg["end_time"]) + w_start
174         seg["_candidate_id"] = candidate_ids[wi]
175         valid.append(seg)
176     except (ValueError, TypeError, KeyError) as e:
177         logger.warning("Dropping segment with unparseable timestamps: %s ?
%s", seg, e)
178     try:
179         os.remove(w_path)
180     except OSError:
181         pass
182     return valid
183
184     ai_segments: List[dict] = []
185     with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
186         futures = [executor.submit(process_candidate_window, wi, cw)
187                     for wi, cw in enumerate(all_candidate_windows)]
188         for future in futures:
189             ai_segments.extend(future.result())
190     try:
191         os.rmdir(handoff_dir)
192     except OSError:
193         pass
194     logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
195
196     # --- Phase 4: DB population (shared pattern) ---
197     segments_data = []
198     for segment in ai_segments:
199         try:
200             start = float(segment["start_time"])
201             end = float(segment["end_time"])
202             dur = end - start
203             shot_count = segment.get("shuttle_exchange_count")
204             try:
205                 shot_count = int(shot_count)
206             except (ValueError, TypeError):
207                 shot_count = max(3, int(dur / 0.8))
208             metadata = {"shot_count": shot_count,
209                        "shot_count_confidence":
segment.get("shot_count_confidence", "Unknown"),
210                        "detector": f"wasb-hybrid-{model_name}"}
211             segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
212             if segment_id:
213                 candidate_id = segment.get("_candidate_id")
214                 if candidate_id is not None:
215                     self.db.link_candidate_to_segment(candidate_id, segment_id)
216                 segments_data.append({"id": segment_id, "start_time": start,
"end_time": end,
217                                     "duration": dur, "metadata": metadata})
218         except Exception as e:
219             logger.warning("Skipping invalid segment struct: %s", e)
220
221     return segments_data
222
223
224     segmenter_registry.register(WasbHybridSegmenter)
225

```

6. user (2026-06-06T15:20:46.467Z)

```

1     """TrackNet-based hybrid play segmenter.
2

```

```

3      Same three-phase shape as HybridYoloSegmenter, but Phase 2 (candidate
4      generation) uses a TrackNetV4 *shuttle* trajectory instead of YOLO *person*
5      motion. TrackNet runs in WSL2 (see backend/detectors/tracknet_runner.py and
6      docs/DECISIONS.md ADR-001); the shuttle trajectory is converted into candidate
7      rally windows, which are then handed to the AI provider for semantic boundary
8      confirmation (Phase 3) and written to the DB (Phase 4).
9
10     Rationale: a tiny, fast, motion-blurred shuttle is exactly what general
11     detectors miss; a purpose-built trajectory model localises it far more
12     precisely, giving cleaner candidate rallies than person-velocity heuristics.
13     """
14
15     import os
16     import time
17     import logging
18     import concurrent.futures
19     from typing import List, Dict, Any
20
21     import cv2
22
23     from backend.indexer.base import BasePlaySegmenter, segmenter_registry
24     from backend.utils.video import get_video_duration, extract_video_chunk
25     from backend.sports import sport_registry
26     from backend.providers import provider_registry
27     from backend.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
28
29     logger = logging.getLogger(__name__)
30
31
32     def _fmt_ts(seconds: float) -> str:
33         seconds = max(0, int(seconds))
34         h, rem = divmod(seconds, 3600)
35         m, s = divmod(rem, 60)
36         return f"{h:02d}:{m:02d}:{s:02d}"
37
38
39     class TrackNetHybridSegmenter(BasePlaySegmenter):
40         @property
41         def name(self) -> str:
42             return "tracknet_hybrid"
43
44         @property
45         def description(self) -> str:
46             return ("Two-stage pipeline: TrackNetV4 shuttle-trajectory detection (WSL2) for
precise "
47                     "candidate rallies + AI provider for high-precision semantic
boundaries.")
48
49         @staticmethod
50         def _probe_fps_width(video_path: str) -> tuple:
51             """Return (fps, frame_width) for a video, with sensible fallbacks."""
52             cap = cv2.VideoCapture(video_path)
53             fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
54             width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
55             cap.release()
56             return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
57
58         def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None,
59                         max_frames: int = None) -> List[Dict[str, Any]]:
60             # --- Sport profile + AI provider wiring (identical to the other segmenters) ---
61             sport_name = self.config.get("sport", "badminton")
62             try:
63                 sport_profile = sport_registry.get(sport_name)
64             except KeyError as e:

```

```

65         logger.error(str(e))
66         return []
67
68         idx_cfg = self.config.get("indexing", {})
69         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
70         model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
71
72         api_key_env_var = f"{provider_name.upper()}_API_KEY"
73         api_key = os.environ.get(api_key_env_var)
74         if not api_key:
75             logger.error(
76                 f"{api_key_env_var} environment variable is not set! "
77                 f"To run the {provider_name} handoff, set your API key. "
78                 f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
79             )
80             return []
81         try:
82             provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
83         except KeyError as e:
84             logger.error(str(e))
85             return []
86
87         video_duration = get_video_duration(video_path)
88         if video_duration <= 0:
89             logger.error("Invalid video duration.")
90             return []
91         fps, frame_width = self._probe_fps_width(video_path)
92
93         # --- TrackNet runner config + thresholds ---
94         tn_cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
95         runner = TrackNetRunner(tn_cfg)
96
97         tn = idx_cfg.get("tracknet", {})
98         velocity_thresh = tn.get("velocity_threshold", 0.02) # normalised shuttle
displacement/sec
99         merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
100         min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
101         handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
102         ai_max_workers = idx_cfg.get("ai_max_workers", 5)
103         log_candidates = idx_cfg.get("log_yolo_candidates", True)
104         store_candidates = idx_cfg.get("store_yolo_candidates", True)
105
106         detection_params = {
107             "detector": "tracknet_hybrid",
108             "velocity_thresh": velocity_thresh,
109             "merge_gap": merge_gap,
110             "min_action_window_duration": min_window_duration,
111             "weights_path": tn_cfg.weights_path,
112             "queue_length": tn_cfg.queue_length,
113         }
114
115         # --- Phase 1: env healthcheck (fail fast with a clear message) ---
116         ok, msg = runner.healthcheck()
117         if not ok:
118             logger.error("TrackNet WSL environment not ready: %s", msg)
119             return []
120         logger.info("TrackNet WSL env OK: %s", msg)
121
122         # --- Phase 2: shuttle trajectory -> candidate rally windows ---
123         # TrackNet processes the whole video in one pass (it carries its own
124         # frame buffering), so unlike the YOLO segmenter we don't pre-chunk here.
125         t_start = time.time()

```



```

126     tn_out_dir = os.path.join("output", "tracknet")
127     csv_path = runner.run_predict(video_path, tn_out_dir)
128     if not csv_path:
129         logger.error("TrackNet produced no trajectory; aborting.")
130         return []
131
132     points = runner.parse_trajectory_csv(csv_path)
133     logger.info("Parsed %d trajectory points (%d visible).",
134                 len(points), sum(1 for p in points if p.visible))
135
136     all_candidate_windows = runner.trajectory_to_action_windows(
137         points, fps=fps, frame_width=frame_width, chunk_start=0.0,
138         velocity_thresh=velocity_thresh, merge_gap=merge_gap,
139         min_window_duration=min_window_duration, chunk_index=0,
140     )
141     logger.info("Phase 2 (TrackNet) completed in %.1fs. Candidate windows: %d",
142                 time.time() - t_start, len(all_candidate_windows))
143
144     if log_candidates:
145         for w in all_candidate_windows:
146             logger.info(
147                 f"[TrackNet rally] {_fmt_ts(w['start'])}-{_fmt_ts(w['end'])} "
148                 f"({w['end'] - w['start']:.1f}s) | frames={w['active_frame_count']}"
149                 f"peak_v={w['peak_velocity']:.3f} mean_v={w['mean_velocity']:.3f}"
150             )
151
152     # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
153     candidate_ids = [None] * len(all_candidate_windows)
154     if store_candidates:
155         for i, w in enumerate(all_candidate_windows):
156             stats = {
157                 "peak_velocity": w["peak_velocity"],
158                 "mean_velocity": w["mean_velocity"],
159                 "active_frame_count": w["active_frame_count"],
160                 "track_id_count": w["track_id_count"],
161             }
162             confidence = min(1.0, w["peak_velocity"])
163             candidate_ids[i] = self.db.add_candidate_segment(
164                 video_id, detector="tracknet_hybrid",
165                 start_time=w["start"], end_time=w["end"],
166                 chunk_index=w["chunk_index"], confidence=confidence,
167                 stats=stats, detection_params=detection_params,
168             )
169
170     if not all_candidate_windows:
171         return []
172
173     # --- Phase 3: AI provider handoff (identical pattern to HybridYoloSegmenter)
174     ---
175     handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
176     os.makedirs(handoff_dir, exist_ok=True)
177     logger.info("Phase 3: %s validation on %d candidate windows...",
178                 provider_name, len(all_candidate_windows))
179
180     def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
181         w_start = max(0, window["start"] - handoff_padding)
182         w_end = min(video_duration, window["end"] + handoff_padding)
183         w_dur = w_end - w_start
184         w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
185         if not extract_video_chunk(video_path, w_start, w_dur, w_path,
186                                   reencode=True):
187             return []
188         prompt = sport_profile.get_prompt(chunk_duration=w_dur)
189         segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,

```

```

188                                     label=f"Handoff {wi+1}")
189         valid = []
190         for seg in segments:
191             try:
192                 seg["start_time"] = float(seg["start_time"]) + w_start
193                 seg["end_time"] = float(seg["end_time"]) + w_start
194                 seg["_candidate_id"] = candidate_ids[wi]
195                 valid.append(seg)
196             except (ValueError, TypeError, KeyError) as e:
197                 logger.warning("Dropping segment with unparseable timestamps: %s ?
%s", seg, e)
198         try:
199             os.remove(w_path)
200         except OSError:
201             pass
202         return valid
203
204     ai_segments: List[dict] = []
205     with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
206         futures = [executor.submit(process_candidate_window, wi, w)
207                     for wi, w in enumerate(all_candidate_windows)]
208         for future in futures:
209             ai_segments.extend(future.result())
210
211     try:
212         os.rmdir(handoff_dir)
213     except OSError:
214         pass
215     logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
216
217     # --- Phase 4: DB population (identical pattern to HybridYoloSegmenter) ---
218     segments_data = []
219     for segment in ai_segments:
220         try:
221             start = float(segment["start_time"])
222             end = float(segment["end_time"])
223             dur = end - start
224             shot_count = segment.get("shuttle_exchange_count")
225             if shot_count is None:
226                 shot_count = max(3, int(dur / 0.8))
227             else:
228                 try:
229                     shot_count = int(shot_count)
230                 except (ValueError, TypeError):
231                     shot_count = max(3, int(dur / 0.8))
232             metadata = {
233                 "shot_count": shot_count,
234                 "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
235                 "detector": f"tracknet-hybrid-{model_name}",
236             }
237             segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
238             if segment_id:
239                 candidate_id = segment.get("_candidate_id")
240                 if candidate_id is not None:
241                     self.db.link_candidate_to_segment(candidate_id, segment_id)
242                 segments_data.append({
243                     "id": segment_id, "start_time": start, "end_time": end,
244                     "duration": dur, "metadata": metadata,
245                 })
246         except Exception as e:
247             logger.warning("Skipping invalid segment struct: %s", e)

```

```

248
249         return segments_data
250
251
252     # Register the segmenter
253     segmenter_registry.register(TrackNetHybridSegmenter)
254

```

7. assistant (2026-06-06T15:20:50.501Z)

Now let me read the remaining three files.

8. user (2026-06-06T15:20:51.882Z)

```

1     """Two-stage rally segmenter: torchvision person detection + ByteTrack tracking
2     (Stage 1, local) feeding an AI provider for semantic boundaries (Stage 2).
3
4     Stage 1 originally used the AGPL-licensed ultralytics YOLO. To keep the hosted
5     service free of copyleft obligations (see docs/MONETIZATION_AUDIT.md and ADR-005),
6     detection now uses torchvision's permissively-licensed (BSD) detectors and the
7     association/tracking that YOLO.track() used to bundle in is provided by
8     supervision's MIT-licensed ByteTrack.
9
10    The registry key stays "yolo_hybrid" for back-compat with existing configs and the
11    DB `detector` tags, even though no YOLO is involved anymore.
12    """
13    import os
14    import time
15    import logging
16    import cv2
17    import torch
18    import concurrent.futures
19    from typing import List, Dict, Any, Tuple
20
21    from backend.indexer.base import BasePlaySegmenter, segmenter_registry
22    from backend.utils.video import get_video_duration, extract_video_chunk
23    from backend.utils.geometry import get_velocity, get_velocity_normalised
24    from backend.sports import sport_registry
25    from backend.providers import provider_registry
26
27    logger = logging.getLogger(__name__)
28
29    # torchvision detection models are trained on COCO where the "person" category is
30    # class 1 (index 0 is the implicit background). NB: ultralytics YOLO used class 0 ?
31    # this off-by-one is the classic gotcha when swapping detectors. Keep it named.
32    _PERSON_CLASS_ID = 1
33
34    # Permissively-licensed torchvision detectors selectable via config
35    # `indexing.detector_model`. All are BSD-licensed (code + COCO weights).
36    _DETECTOR_FACTORIES = {
37        "fasterrcnn_resnet50_fpn": "fasterrcnn_resnet50_fpn",
38        "fasterrcnn_mobilenet_v3_large_fpn": "fasterrcnn_mobilenet_v3_large_fpn",
39        "retinanet_resnet50_fpn": "retinanet_resnet50_fpn",
40    }
41    _DEFAULT_DETECTOR = "fasterrcnn_resnet50_fpn"
42
43
44    def _fmt_ts(seconds: float) -> str:
45        """Format seconds as HH:MM:SS for cross-verifying a timestamp against the video."""
46        seconds = max(0, int(seconds))
47        h, rem = divmod(seconds, 3600)
48        m, s = divmod(rem, 60)
49        return f"{h:02d}:{m:02d}:{s:02d}"
50
51

```

```

52     class HybridYoloSegmenter(BasePlaySegmenter):
53         def __init__(self, db, config):
54             super().__init__(db, config)
55             self.model = None
56             self.device = None
57
58         @property
59         def name(self) -> str:
60             return "yolo_hybrid"
61
62         @property
63         def description(self) -> str:
64             return ("Two-stage pipeline: torchvision person detection + ByteTrack for fast "
65                     "candidate filtering, then an AI provider for high-precision semantic
66 boundaries.")
67
68         def _lazy_load_model(self):
69             if self.model is not None:
70                 return
71
72             # Imported lazily so the module imports without torchvision present and so
73             # the test suite (which pre-sets self.model) never pulls in model weights.
74             from torchvision.models import detection as tv_detection
75
76             idx_cfg = self.config.get("indexing", {})
77             use_gpu = idx_cfg.get("use_gpu", True)
78             self.device = "cuda" if use_gpu and torch.cuda.is_available() else "cpu"
79             if use_gpu and not torch.cuda.is_available():
80                 logger.warning("use_gpu is True but CUDA is not available. Falling back to
81 CPU.")
82
83             model_name = idx_cfg.get("detector_model", _DEFAULT_DETECTOR)
84             if model_name not in _DETECTOR_FACTORIES:
85                 logger.warning(f"Unknown detector_model '{model_name}'. Falling back to
86 { _DEFAULT_DETECTOR}.")
87             model_name = _DEFAULT_DETECTOR
88
89             builder = getattr(tv_detection, _DETECTOR_FACTORIES[model_name])
90             logger.info(f"Loading torchvision detector '{model_name}' on {self.device}...")
91             # weights="DEFAULT" pulls the best available COCO-pretrained checkpoint.
92             self.model = builder(weights="DEFAULT")
93             self.model.eval()
94             self.model.to(self.device)
95
96         def _make_tracker(self, fps: float):
97             """Create a fresh ByteTrack tracker for one chunk (track IDs are chunk-local).
98
99             Imported lazily so tests can mock this method without supervision installed.
100             """
101             import supervision as sv
102             frame_rate = max(1, int(round(fps)))
103             return sv.ByteTrack(frame_rate=frame_rate)
104
105         def _detect_and_track(self, frame, tracker, score_thresh: float) -> List[Tuple[int,
106 Tuple[float, float, float, float]]]:
107             """Detect persons in a frame and assign persistent track IDs.
108
109             Returns a list of (track_id, bbox) where bbox is (x1, y1, x2, y2) in pixels.
110             This replaces the old ultralytics ``model.track(...)`` call, which bundled
111             detection AND association: torchvision does the detection, supervision's
112             ByteTrack does the association.
113             """
114             import numpy as np
115             import supervision as sv

```

```

113         # cv2 frames are BGR uint8 HxWxC; torchvision wants RGB float [0,1] CxHxW.
114         rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
115         tensor = torch.from_numpy(rgb).permute(2, 0,
1)         .float().div(255.0).to(self.device)
116         with torch.no_grad():
117             outputs = self.model([tensor])
118
119         out = outputs[0]
120         boxes = out["boxes"].detach().cpu().numpy()
121         labels = out["labels"].detach().cpu().numpy()
122         scores = out["scores"].detach().cpu().numpy()
123
124         # Keep only confident person detections.
125         mask = (labels == _PERSON_CLASS_ID) & (scores >= score_thresh)
126         if not mask.any():
127             return []
128
129         # ByteTrack requires confidence to be populated (it filters on it internally).
130         dets = sv.Detections(xyxy=boxes[mask], confidence=scores[mask],
class_id=labels[mask])
131         tracked = tracker.update_with_detections(dets)
132
133         result: List[Tuple[int, Tuple[float, float, float, float]]] = []
134         for i in range(len(tracked)):
135             tid = tracked.tracker_id[i]
136             if tid is None:
137                 continue
138             x1, y1, x2, y2 = tracked.xyxy[i]
139             result.append((int(tid), (float(x1), float(y1), float(x2), float(y2))))
140         return result
141
142     def _extract_action_windows_from_chunk(self, chunk_path: str, chunk_start: float,
143                                           velocity_thresh: float, merge_gap: float,
144                                           frame_skip: int = 3,
145                                           tracker_config: str = "bytetrack.yaml",
146                                           chunk_index: int = None) -> List[Dict[str,
Any]]:
147         """
148         Runs person detection + tracking on a video chunk and extracts high-motion
149         action windows. Returns a list of dicts in the full video's timeframe, each
with:
150             start, end (seconds), active_frame_count, peak_velocity, mean_velocity,
151             track_id_count, chunk_index.
152
153         Args:
154             frame_skip: Process every Nth frame (1 = every frame). Higher values
155                 dramatically reduce inference cost with minimal quality loss.
156             tracker_config: Retained for back-compat (was the YOLO tracker yaml); the
157                 tracker is now supervision ByteTrack and this is unused.
158             chunk_index: Index of the source chunk (recorded on each window for
traceability).
159         """
160         cap = cv2.VideoCapture(chunk_path)
161         if not cap.isOpened():
162             logger.error(f"Could not open {chunk_path}")
163             return []
164
165         fps = cap.get(cv2.CAP_PROP_FPS)
166         if fps <= 0:
167             fps = 30.0
168
169         frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
170         if frame_width <= 0:
171             frame_width = 1280.0 # Sensible fallback
172

```

```

173     # Effective sampling rate after frame-skipping
174     effective_fps = fps / max(frame_skip, 1)
175
176     # A fresh tracker per chunk ? track IDs only need to be consistent within a
177     # chunk (windows are computed per chunk, then merged across chunks by time).
178     tracker = self._make_tracker(effective_fps)
179     score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
0.5)
180
181     # Per active frame we keep (frame_idx, peak_velocity, set_of_active_track_ids)
182     # so windows can be enriched with motion stats for logging + fine-tuning.
183     active_frames = []
184     frame_idx = 0
185     track_history = {}
186
187     while True:
188         ret, frame = cap.read()
189         if not ret:
190             break
191
192         # Skip frames for performance ? at 30fps, every 3rd frame still
193         # gives ~10 samples/second, plenty for human-scale velocity.
194         if frame_idx % max(frame_skip, 1) != 0:
195             frame_idx += 1
196             continue
197
198         # Detect persons + assign persistent track IDs.
199         detections = self._detect_and_track(frame, tracker, score_thresh)
200
201         frame_peak_velocity = 0.0
202         frame_active_ids = set()
203         current_ids = set()
204
205         for track_id, bbox in detections:
206             current_ids.add(track_id)
207
208             if track_id in track_history:
209                 prev_bbox = track_history[track_id]
210                 velocity = get_velocity_normalised(prev_bbox, bbox, effective_fps,
frame_width)
211
212                 if velocity > velocity_thresh:
213                     frame_peak_velocity = max(frame_peak_velocity, velocity)
214                     frame_active_ids.add(track_id)
215
216             track_history[track_id] = bbox
217
218         # Prune stale IDs that are no longer visible ? prevents false
219         # velocity spikes when the tracker reassigns an old ID.
220         track_history = {k: v for k, v in track_history.items() if k in current_ids}
221
222         if frame_active_ids:
223             active_frames.append((frame_idx, frame_peak_velocity, frame_active_ids))
224
225         frame_idx += 1
226
227     cap.release()
228
229     # Group active frames into continuous windows
230     if not active_frames:
231         return []
232
233     max_gap_frames = int(merge_gap * fps)
234
235     def _finalize(group: List[Tuple[int, float, set]]) -> Dict[str, Any]:
236         velocities = [v for _, v, _ in group]

```

```

236         track_ids = set().union(*[ids for _, _, ids in group])
237     return {
238         "start": group[0][0] / fps + chunk_start,
239         "end": group[-1][0] / fps + chunk_start,
240         "active_frame_count": len(group),
241         "peak_velocity": max(velocities),
242         "mean_velocity": sum(velocities) / len(velocities),
243         "track_id_count": len(track_ids),
244         "chunk_index": chunk_index,
245     }
246
247     windows = []
248     group = [active_frames[0]]
249     for af in active_frames[1:]:
250         if af[0] - group[-1][0] <= max_gap_frames:
251             group.append(af)
252         else:
253             windows.append(_finalize(group))
254             group = [af]
255     windows.append(_finalize(group))
256
257     # Filter out extremely short action windows
258     min_window_duration = self.config.get("indexing",
259     {}).get("min_action_window_duration", 2.0)
259     windows = [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
260
261     return windows
262
263     def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
264         self._lazy_load_model()
265
266         # Get sport profile config
267         sport_name = self.config.get("sport", "badminton")
268         try:
269             sport_profile = sport_registry.get(sport_name)
270         except KeyError as e:
271             logger.error(str(e))
272         return []
273
274         # Get AI provider and model config
275         idx_cfg = self.config.get("indexing", {})
276         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
277         model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
278
279         # Get appropriate API key based on the provider
280         api_key_env_var = f"{provider_name.upper()}_API_KEY"
281         api_key = os.environ.get(api_key_env_var)
282         if not api_key:
283             logger.error(
284                 f"{api_key_env_var} environment variable is not set! "
285                 f"To run the {provider_name} segmenter, set your API key. "
286                 f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
287             )
288         return []
289
290         try:
291             provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
292         except KeyError as e:
293             logger.error(str(e))
294         return []
295

```

```

296         logger.info(f"Initializing Hybrid Segmenter (torchvision + ByteTrack) for video:
{video_path}")
297
298         video_duration = get_video_duration(video_path)
299         if video_duration <= 0:
300             logger.error("Invalid video duration.")
301             return []
302
303         chunk_duration = idx_cfg.get("chunk_duration", 300)
304         chunk_overlap = idx_cfg.get("chunk_overlap", 30)
305         velocity_thresh = idx_cfg.get("yolo_velocity_threshold", 0.15) # Fraction of
frame width per second
306         merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
307         frame_skip = idx_cfg.get("yolo_frame_skip", 3)
308         tracker_config = idx_cfg.get("yolo_tracker", "bytetrack.yaml")
309         handoff_padding = idx_cfg.get("ai_handoff_padding",
idx_cfg.get("gemini_handoff_padding", 2.0))
310         ai_max_workers = idx_cfg.get("ai_max_workers", idx_cfg.get("gemini_max_workers",
5))
311         log_candidates = idx_cfg.get("log_yolo_candidates", True)
312         store_candidates = idx_cfg.get("store_yolo_candidates", True)
313
314         # Snapshot of the params that produced these detections ? stored alongside each
315         # candidate so a fine-tuning run can reproduce / correlate against them.
316         detection_params = {
317             "velocity_thresh": velocity_thresh,
318             "merge_gap": merge_gap,
319             "frame_skip": frame_skip,
320             "tracker": tracker_config,
321             "detector_model": idx_cfg.get("detector_model", _DEFAULT_DETECTOR),
322             "detector_score_threshold": idx_cfg.get("detector_score_threshold", 0.5),
323             "min_action_window_duration": idx_cfg.get("min_action_window_duration",
2.0),
324         }
325
326         chunks_dir = os.path.join("output", "chunks_yolo")
327         os.makedirs(chunks_dir, exist_ok=True)
328
329         step = chunk_duration - chunk_overlap
330         chunk_starts = []
331         t = 0.0
332         while t < video_duration:
333             chunk_starts.append(t)
334             t += step
335
336         num_chunks = len(chunk_starts)
337         logger.info(f"Video duration: {video_duration:.1f}s. Splitting into {num_chunks}
chunks.")
338
339         all_candidate_windows = []
340
341         t_start = time.time()
342         prefetch_workers = idx_cfg.get("yolo_prefetch_workers", 2)
343
344         if num_chunks > 1 and prefetch_workers > 0:
345             # --- PIPELINED PROCESSING ---
346             # GPU inference must stay sequential (single GPU), but ffmpeg chunk
347             # extraction is pure I/O. We pre-extract upcoming chunks in background
348             # threads so extraction of chunk N+1 overlaps with inference on chunk N.
349             logger.info(f"Running detection Phase 2 with pipelined extraction
({prefetch_workers} prefetch workers)...")
350
351             extract_executor =
concurrent.futures.ThreadPoolExecutor(max_workers=prefetch_workers)
352

```



```

353     def _extract_chunk(ci: int, cs: float) -> Tuple[int, float, str, bool]:
354         chunk_end = min(cs + chunk_duration, video_duration)
355         actual_dur = chunk_end - cs
356         chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
357         success = extract_video_chunk(video_path, cs, actual_dur, chunk_path)
358         return (ci, cs, chunk_path, success)
359
360     # Submit all extractions upfront ? they'll queue and run as workers free up
361     extract_futures = [
362         extract_executor.submit(_extract_chunk, ci, cs)
363         for ci, cs in enumerate(chunk_starts)
364     ]
365
366     # Process results in order: wait for extraction, then run inference on GPU
367     for future in extract_futures:
368         ci, chunk_start, chunk_path, success = future.result()
369         chunk_end = min(chunk_start + chunk_duration, video_duration)
370         logger.info(f"Processing Chunk {ci+1}/{num_chunks} ({chunk_start:.1f}s -
{chunk_end:.1f}s)...")
371
372         if not success:
373             logger.warning(f"Chunk {ci+1} extraction failed. Skipping.")
374             continue
375
376         windows = self._extract_action_windows_from_chunk(
377             chunk_path, chunk_start, velocity_thresh, merge_gap,
378             frame_skip=frame_skip, tracker_config=tracker_config, chunk_index=ci
379         )
380         logger.info(f"Found {len(windows)} candidate action windows in chunk
{ci+1}.")
381         all_candidate_windows.extend(windows)
382
383         try:
384             os.remove(chunk_path)
385         except Exception:
386             pass
387
388     extract_executor.shutdown(wait=False)
389     else:
390         # --- SEQUENTIAL PROCESSING (single chunk or prefetch disabled) ---
391         for ci, chunk_start in enumerate(chunk_starts):
392             chunk_end = min(chunk_start + chunk_duration, video_duration)
393             actual_dur = chunk_end - chunk_start
394             chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
395
396             logger.info(f"Processing Chunk {ci+1}/{num_chunks} ({chunk_start:.1f}s -
{chunk_end:.1f}s)...")
397             success = extract_video_chunk(video_path, chunk_start, actual_dur,
chunk_path)
398
399             if not success:
400                 continue
401
402             windows = self._extract_action_windows_from_chunk(
403                 chunk_path, chunk_start, velocity_thresh, merge_gap,
404                 frame_skip=frame_skip, tracker_config=tracker_config, chunk_index=ci
405             )
406             logger.info(f"Found {len(windows)} candidate action windows in chunk
{ci+1}.")
407             all_candidate_windows.extend(windows)
408
409             try:
410                 os.remove(chunk_path)
411             except Exception:
412                 pass

```

```

413 # Deduplicate overlapping candidate windows across chunks (chunks overlap by
414 # chunk_overlap, so the same rally can surface in two adjacent chunks).
415 def _merge_stats(a: Dict[str, Any], b: Dict[str, Any]) -> Dict[str, Any]:
416     fa, fb = a["active_frame_count"], b["active_frame_count"]
417     total = fa + fb or 1
418     return {
419         "start": a["start"],
420         "end": max(a["end"], b["end"]),
421         "active_frame_count": fa + fb,
422         "peak_velocity": max(a["peak_velocity"], b["peak_velocity"]),
423         # Frame-weighted mean so longer sub-windows dominate proportionally.
424         "mean_velocity": (a["mean_velocity"] * fa + b["mean_velocity"] * fb) /
total,
425         "track_id_count": max(a["track_id_count"], b["track_id_count"]),
426         "chunk_index": a["chunk_index"],
427     }
428
429 if all_candidate_windows:
430     all_candidate_windows.sort(key=lambda w: w["start"])
431     merged_windows = [all_candidate_windows[0]]
432     for current in all_candidate_windows[1:]:
433         last = merged_windows[-1]
434         if current["start"] <= last["end"]: # Overlap
435             merged_windows[-1] = _merge_stats(last, current)
436         else:
437             merged_windows.append(current)
438     all_candidate_windows = merged_windows
439
440 logger.info(f"Phase 2 tracking completed in {time.time() - t_start:.1f}s.")
441 logger.info(f"Total candidate windows: {len(all_candidate_windows)}")
442
443 # Per-rally console log (final, full-video timestamps) for video cross-
verification.
444 if log_candidates:
445     for w in all_candidate_windows:
446         logger.info(
447             f"[rally] {_fmt_ts(w['start'])}-{_fmt_ts(w['end'])} "
448             f"({w['end'] - w['start']:.1f}s) | frames={w['active_frame_count']} "
449             f"peak_v={w['peak_velocity']:.3f} mean_v={w['mean_velocity']:.3f} "
450             f"tracks={w['track_id_count']}"
451         )
452
453 # Persist raw candidates for the fine-tuning dataset. Map window index ->
candidate_id
454 # so confirmed AI segments can be linked back in Phase 4.
455 candidate_ids = [None] * len(all_candidate_windows)
456 if store_candidates:
457     for i, w in enumerate(all_candidate_windows):
458         stats = {
459             "peak_velocity": w["peak_velocity"],
460             "mean_velocity": w["mean_velocity"],
461             "active_frame_count": w["active_frame_count"],
462             "track_id_count": w["track_id_count"],
463         }
464         # Use peak velocity (capped at 1.0) as a coarse generic confidence
score.
465         confidence = min(1.0, w["peak_velocity"])
466         candidate_ids[i] = self.db.add_candidate_segment(
467             video_id, detector="yolo_hybrid",
468             start_time=w["start"], end_time=w["end"],
469             chunk_index=w["chunk_index"], confidence=confidence,
470             stats=stats, detection_params=detection_params,
471         )
472

```

```

473         try:
474             os.rmdir(chunks_dir)
475         except Exception:
476             pass
477
478         # Phase 3: AI Provider Handoff
479         gemini_segments = []
480         gemini_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
481         os.makedirs(gemini_dir, exist_ok=True)
482
483         logger.info(f"Starting Phase 3: AI Provider ({provider_name}) Validation on
{len(all_candidate_windows)} candidate windows...")
484
485         def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
486             w_start = max(0, window["start"] - handoff_padding)
487             w_end = min(video_duration, window["end"] + handoff_padding)
488             w_dur = w_end - w_start
489
490             w_path = os.path.join(gemini_dir, f"handoff_{video_id}_{wi}.mp4")
491             # Re-encode short handoff clips to avoid keyframe-boundary drift
492             success = extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True)
493             if not success:
494                 return []
495
496             prompt = sport_profile.get_prompt(chunk_duration=w_dur)
497             segments, metrics = provider.analyze_video(w_path, prompt,
chunk_duration=w_dur, label=f"Handoff {wi+1}")
498
499             # Coordinate mapping ? shift local timestamps back to full-video time
500             valid_segments = []
501             for segment in segments:
502                 try:
503                     segment["start_time"] = float(segment["start_time"]) + w_start
504                     segment["end_time"] = float(segment["end_time"]) + w_start
505                     # Stamp the source candidate so Phase 4 can link
506                     segment["_candidate_id"] = candidate_ids[wi]
507                     valid_segments.append(segment)
508                 except (ValueError, TypeError, KeyError) as e:
509                     logger.warning(f"Dropping segment with unparseable timestamps:
{segment} ? {e}")
510
511             try:
512                 os.remove(w_path)
513             except OSError:
514                 pass
515
516             return valid_segments
517
518         with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
519             futures = [executor.submit(process_candidate_window, wi, w) for wi, w in
enumerate(all_candidate_windows)]
520             # Iterate in submission order to preserve temporal ordering
521             for future in futures:
522                 gemini_segments.extend(future.result())
523
524         try:
525             os.rmdir(gemini_dir)
526         except OSError:
527             pass
528
529         logger.info(f"Phase 3 completed. AI confirmed {len(gemini_segments)} semantic
play segments.")

```

```

530
531     # Phase 4: DB Population
532     segments_data = []
533     for segment in gemini_segments:
534         try:
535             start = float(segment["start_time"])
536             end = float(segment["end_time"])
537             dur = end - start
538
539             shot_count = segment.get("shuttle_exchange_count")
540             if shot_count is None: shot_count = max(3, int(dur / 0.8))
541             else:
542                 try: shot_count = int(shot_count)
543                 except (ValueError, TypeError): shot_count = max(3, int(dur / 0.8))
544
545             metadata = {
546                 "shot_count": shot_count,
547                 "shot_count_confidence": segment.get("shot_count_confidence",
548 "Unknown"),
549                 "detector": f"yolo-hybrid-{model_name}"
550             }
551             segment_id = self.db.add_play_segment(video_id, start, end, None, None,
552 metadata)
553             if segment_id:
554                 # Link back to the candidate that produced this segment. A candidate
555                 # may yield multiple AI segments (one-to-many); confirmed_segment_id
556                 # keeps
557                 # the last one ? the link table records "this candidate was
558                 confirmed".
559                 candidate_id = segment.get("_candidate_id")
560                 if candidate_id is not None:
561                     self.db.link_candidate_to_segment(candidate_id, segment_id)
562                 segments_data.append({
563                     "id": segment_id,
564                     "start_time": start,
565                     "end_time": end,
566                     "duration": dur,
567                     "metadata": metadata
568                 })
569             except Exception as e:
570                 logger.warning(f"Skipping invalid segment struct: {e}")
571
572     return segments_data
573
574 # Register the segmenter
575 segmenter_registry.register(HybridYoloSegmenter)
576
577

```

9. user (2026-06-06T15:20:51.934Z)

```

1     import cv2
2     import time
3     import numpy as np
4     import random
5     from typing import List, Dict, Any, Tuple
6     from backend.database import Database
7     from backend.indexer.base import BasePlaySegmenter, segmenter_registry
8
9     class MotionPlaySegmenter(BasePlaySegmenter):
10         @property
11         def name(self) -> str:
12             return "motion"
13
14         @property

```

```

15     def description(self) -> str:
16         return "Traditional CV pixel-motion heuristics tracking frame differences."
17
18     def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
19         """
20         Processes a video to detect play segments using motion heuristics,
21         populates SQLite with detected segments, and indexes detailed events.
22         """
23         if not player_pool:
24             player_pool = ["Avi", "Suresh", "Ramesh", "Deepak"]
25
26         cap = cv2.VideoCapture(video_path)
27         if not cap.isOpened():
28             print(f"[ERROR] Segmenter could not open video: {video_path}")
29             return []
30
31         fps = cap.get(cv2.CAP_PROP_FPS)
32         total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
33         duration = total_frames / fps if fps > 0 else 0
34
35         # Sub-sample settings (5 FPS for analysis)
36         analysis_fps = 5.0
37         frame_interval = max(1, int(fps / analysis_fps))
38
39         idx_cfg = self.config.get("indexing", {})
40         motion_threshold = idx_cfg.get("motion_threshold", 0.08)
41         min_segment_duration = idx_cfg.get("min_segment_duration", 3.0)
42         dead_time_merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
43
44         prev_gray = None
45         motion_signals = []
46         timestamps = []
47
48         frame_idx = 0
49         processed_count = 0
50         t_start = time.time()
51         # Emit a progress line roughly every 5% of the video so long runs are
52         # observable instead of silent (a full 1080p match is ~tens of minutes).
53         progress_step = max(1, total_frames // 20) if total_frames > 0 else 0
54         next_progress = progress_step
55         print(f"[motion] start: {total_frames} frames @ {fps:.1f}fps "
56               f"(~{duration/60:.1f} min), analyzing every {frame_interval}th frame",
flush=True)
57         while True:
58             # Skip decoding for intermediate frames using cap.grab() for high
performance
59             for _ in range(frame_interval - 1):
60                 if not cap.grab():
61                     break
62                 frame_idx += 1
63
64             ret, frame = cap.read()
65             if not ret:
66                 break
67
68             # Process frame
69             gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
70             # Apply Gaussian Blur to smooth noise
71             gray = cv2.GaussianBlur(gray, (21, 21), 0)
72
73             t = frame_idx / fps
74             timestamps.append(t)
75
76             if prev_gray is not None:

```

```

77         # Frame absolute difference (highly optimized motion tracking)
78         diff = cv2.absdiff(prev_gray, gray)
79         _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
80
81         # Dilate to fill gaps
82         thresh = cv2.dilate(thresh, None, iterations=2)
83
84         # Motion ratio
85         motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
86         motion_signals.append(motion_ratio)
87     else:
88         motion_signals.append(0.0)
89
90     prev_gray = gray
91     frame_idx += 1
92     processed_count += 1
93
94     if progress_step and frame_idx >= next_progress:
95         pct = 100.0 * frame_idx / total_frames
96         elapsed = time.time() - t_start
97         eta = elapsed / (frame_idx / total_frames) - elapsed if frame_idx else 0
98         print(f"[motion] {pct:5.1f}% ? {frame_idx}/{total_frames} frames, "
99               f"{processed_count} analyzed, {elapsed:.0f}s elapsed, ~{eta:.0f}s
left", flush=True)
100         next_progress += progress_step
101
102         if max_frames is not None and processed_count >= max_frames:
103             print(f"[DEBUG] Reached max_frames limit of {max_frames}. Halting
indexer frame parsing.")
104             break
105
106         if frame_idx >= total_frames:
107             break
108
109     cap.release()
110
111     # Smooth signal using a moving average
112     window_size = 5
113     smoothed = []
114     for i in range(len(motion_signals)):
115         start = max(0, i - window_size // 2)
116         end = min(len(motion_signals), i + window_size // 2 + 1)
117         smoothed.append(sum(motion_signals[start:end]) / (end - start))
118
119     # Detect active play boundaries
120     in_segment = False
121     segment_start = None
122     raw_segments: List[Tuple[float, float]] = []
123
124     for i, t in enumerate(timestamps):
125         signal = smoothed[i]
126         if signal > motion_threshold:
127             if not in_segment:
128                 in_segment = True
129                 segment_start = t
130             else:
131                 if in_segment:
132                     in_segment = False
133                     raw_segments.append((segment_start, t))
134
135     # Handle video ending while in active segment
136     if in_segment:
137         raw_segments.append((segment_start, timestamps[-1]))
138

```

```

139     # Post-Processing: 1. Merge segments separated by less than dead_time_merge_gap
140     merged_segments: List[Tuple[float, float]] = []
141     if raw_segments:
142         curr_start, curr_end = raw_segments[0]
143         for next_start, next_end in raw_segments[1:]:
144             if next_start - curr_end < dead_time_merge_gap:
145                 # Merge
146                 curr_end = next_end
147             else:
148                 merged_segments.append((curr_start, curr_end))
149                 curr_start, curr_end = next_start, next_end
150         merged_segments.append((curr_start, curr_end))
151
152     # Post-Processing: 2. Filter segments shorter than min_segment_duration
153     final_segments: List[Tuple[float, float]] = [
154         r for r in merged_segments if (r[1] - r[0]) >= min_segment_duration
155     ]
156
157     # Populate SQLite DB with segments and metadata events
158     segments_data = []
159     for start, end in final_segments:
160         # Assign Server and Receiver randomly from pool
161         server, receiver = random.sample(player_pool, 2)
162
163         # Formulate dynamic rich tags
164         dur = end - start
165         metadata = {
166             "shot_count": random.randint(4, 25),
167             "intensity": "high" if dur > 12 else "medium",
168             "avg_speed_kmh": round(random.uniform(40, 85), 1)
169         }
170
171         segment_id = self.db.add_play_segment(video_id, start, end, server,
172 receiver, metadata)
173
174         if segment_id:
175             # Add sub-segment events
176             # A: Serve Event (always at start)
177             serve_foul = random.random() < 0.15 # 15% chance of serve foul
178             self.db.add_event(
179                 segment_id=segment_id,
180                 timestamp=start + 0.5,
181                 event_type="serve",
182                 player=server,
183                 details={"status": "foul" if serve_foul else "valid", "type":
184 "low_serve" if random.random() > 0.5 else "high_serve"}
185             )
186
187             # B: Smash Events (1 to 3 random smashes per segment)
188             smashes_count = random.randint(0, 3)
189             for _ in range(smashes_count):
190                 smash_time = random.uniform(start + 1.5, end - 1.0)
191                 smasher = random.choice([server, receiver])
192                 self.db.add_event(
193                     segment_id=segment_id,
194                     timestamp=round(smash_time, 2),
195                     event_type="smash",
196                     player=smasher,
197                     details={"speed_kmh": round(random.uniform(120, 250), 1)}
198                 )
199
200             # C: Segment Termination Event (foul/winner)
201             foul_types = ["out_of_bounds", "net_touch", "double_hit", "fault"]
202             is_foul = random.random() > 0.4
203             error_player = random.choice([server, receiver])

```

```

202
203         self.db.add_event(
204             segment_id=segment_id,
205             timestamp=end - 0.2,
206             event_type="foul" if is_foul else "winner",
207             player=error_player,
208             details={"type": random.choice(foul_types)} if is_foul else
{"shot_type": "drop" if random.random() > 0.5 else "smash"}
209         )
210
211         # Store segment representation to return to caller
212         segments_data.append({
213             "id": segment_id,
214             "start_time": start,
215             "end_time": end,
216             "duration": dur,
217             "server": server,
218             "receiver": receiver,
219             "metadata": metadata
220         })
221
222         return segments_data
223
224     # Register the segmenter
225     segmenter_registry.register(MotionPlaySegmenter)
226

```

10. user (2026-06-06T15:20:52.389Z)

```

1     import os
2     import time
3     import json
4     import logging
5     import subprocess
6     import concurrent.futures
7     from typing import List, Dict, Any, Tuple
8     from backend.database import Database
9     from backend.indexer.base import BasePlaySegmenter, segmenter_registry
10    from backend.utils.video import get_video_duration, extract_video_chunk
11    from backend.sports import sport_registry
12    from backend.providers import provider_registry
13
14    logger = logging.getLogger(__name__)
15
16    class GeminiPlaySegmenter(BasePlaySegmenter):
17        @property
18        def name(self) -> str:
19            return "gemini"
20
21        @property
22        def description(self) -> str:
23            return "High-precision AI-powered semantic play segment detection using Google
Gemini Multimodal API."
24
25        def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
26            """
27            Processes a video to detect play segments using Google Gemini Multimodal Video
API,
28            populates SQLite, and indexes detailed events.
29            """
30            # Get sport profile config
31            sport_name = self.config.get("sport", "badminton")
32            try:
33                sport_profile = sport_registry.get(sport_name)

```



```

34         except KeyError as e:
35             logger.error(str(e))
36             return []
37
38         # Get AI provider and model config
39         idx_cfg = self.config.get("indexing", {})
40         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
41         model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
42
43         # Get appropriate API key based on the provider
44         api_key_env_var = f"{provider_name.upper()}_API_KEY"
45         api_key = os.environ.get(api_key_env_var)
46         if not api_key:
47             logger.error(
48                 f"{api_key_env_var} environment variable is not set! "
49                 f"To run the {provider_name} segmenter, set your API key. "
50                 f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
51             )
52             return []
53
54         try:
55             provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
56         except KeyError as e:
57             logger.error(str(e))
58             return []
59
60         # A: Check for debug_duration to trim the video first
61         debug_duration = self.config.get("indexing", {}).get("debug_duration")
62         video_to_upload = video_path
63         is_trimmed = False
64         temp_trimmed_path = None
65         is_compressed = False
66         temp_compressed_path = None
67
68         if debug_duration:
69             logger.info(f"DEBUG FEATURE: Trimming video to first {debug_duration}
seconds for fast processing...")
70             temp_trimmed_path = os.path.join("output", f"temp_trimmed_{video_id}.mp4")
71             os.makedirs("output", exist_ok=True)
72             if os.path.exists(temp_trimmed_path):
73                 try:
74                     os.remove(temp_trimmed_path)
75                 except Exception:
76                     pass
77
78             cmd = [
79                 "ffmpeg", "-y",
80                 "-ss", "0",
81                 "-t", str(debug_duration),
82                 "-i", video_path,
83                 "-c", "copy",
84                 temp_trimmed_path
85             ]
86             logger.debug(f"Running command: {' '.join(cmd)}")
87             try:
88                 subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
89                 logger.info(f"Trimmed video successfully created at:
{temp_trimmed_path}")
90                 video_to_upload = temp_trimmed_path
91                 is_trimmed = True
92             except Exception as e:

```

```

93         logger.warning(f"Failed to trim video using ffmpeg: {e}. Falling back to
full video.")
94
95
96
97     # Helper to clean up local temp files
98     def cleanup_temp_files():
99         if is_trimmed and temp_trimmed_path and os.path.exists(temp_trimmed_path):
100             try:
101                 os.remove(temp_trimmed_path)
102             except Exception:
103                 pass
104
105     # 1. Determine actual video duration
106     video_duration = get_video_duration(video_to_upload)
107     if video_duration > 0:
108         logger.info(f"Detected video duration: {video_duration:.1f}s")
109     else:
110         logger.warning("Could not detect video duration. Prompt will not include
duration context.")
111
112     # 2. Decide whether to use chunked processing
113     idx_cfg = self.config.get("indexing", {})
114     chunk_duration = idx_cfg.get("chunk_duration", 300)    # 5 minutes default
115     chunk_overlap = idx_cfg.get("chunk_overlap", 30)       # 30 seconds overlap
116     use_chunking = video_duration > 0 and video_duration > chunk_duration
117
118     ai_segments = []
119     chunks_dir = os.path.join("output", "chunks")
120
121     if use_chunking:
122         # --- CHUNKED PROCESSING ---
123         step = chunk_duration - chunk_overlap
124         chunk_starts = []
125         t = 0.0
126         while t < video_duration:
127             chunk_starts.append(t)
128             t += step
129         num_chunks = len(chunk_starts)
130         logger.info(
131             f"Video is {video_duration:.1f}s ? splitting into {num_chunks}
overlapping chunks "
132             f"({chunk_duration}s each, {chunk_overlap}s overlap).")
133         )
134
135         os.makedirs(chunks_dir, exist_ok=True)
136
137         def process_single_chunk(ci: int, chunk_start: float) -> Tuple[str, list,
dict]:
138             chunk_end = min(chunk_start + chunk_duration, video_duration)
139             actual_chunk_dur = chunk_end - chunk_start
140             chunk_label = f"Chunk {ci+1}/{num_chunks}"
141             chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
142
143             # Extract chunk with ffmpeg
144             logger.info(f"Extracting {chunk_label}: {chunk_start:.1f}s -
{chunk_end:.1f}s ({actual_chunk_dur:.1f}s)")
145             success = extract_video_chunk(video_to_upload, chunk_start,
actual_chunk_dur, chunk_path)
146             if not success:
147                 logger.error(f"Failed to extract {chunk_label}. Skipping.")
148                 return chunk_label, [], {}
149
150             # Measure chunk size
151             try:

```

```

152         chunk_size_mb = os.path.getsize(chunk_path) / (1024 * 1024)
153     except Exception:
154         chunk_size_mb = 0.0
155
156     # Get prompt from sport profile
157     prompt = sport_profile.get_prompt(chunk_duration=actual_chunk_dur)
158
159     # Process this chunk through the provider
160     chunk_segments, metrics = provider.analyze_video(
161         chunk_path, prompt, chunk_duration=actual_chunk_dur,
label=chunk_label
162     )
163     metrics["file_size_mb"] = chunk_size_mb
164     metrics["segments_found"] = len(chunk_segments)
165
166     # Offset timestamps back to full-video coordinates
167     for segment in chunk_segments:
168         if "start_time" in segment:
169             try:
170                 segment["start_time"] = float(segment["start_time"]) +
chunk_start
171             except (ValueError, TypeError):
172                 pass
173         if "end_time" in segment:
174             try:
175                 segment["end_time"] = float(segment["end_time"]) +
chunk_start
176             except (ValueError, TypeError):
177                 pass
178
179     logger.info(f"{chunk_label} returned {len(chunk_segments)} play
segments.")
180
181     # Clean up chunk file
182     try:
183         os.remove(chunk_path)
184     except Exception:
185         pass
186
187     return chunk_label, chunk_segments, metrics
188
189 # Run chunks in parallel
190 all_metrics = {}
191 logger.info("Starting parallel processing with up to 5 concurrent
workers...")
192 t_chunking_start = time.time()
193 with concurrent.futures.ThreadPoolExecutor(max_workers=5) as executor:
194     futures = [executor.submit(process_single_chunk, ci, start) for ci,
start in enumerate(chunk_starts)]
195     for future in concurrent.futures.as_completed(futures):
196         label, segments, metrics = future.result()
197         ai_segments.extend(segments)
198         if metrics:
199             all_metrics[label] = metrics
200
201 # Print Summary Log
202 summary_lines = [
203     "",
204     "=" * 60,
205     "                GEMINI CHUNKING SUMMARY",
206     "=" * 60,
207     f" Total time: {time.time() - t_chunking_start:.1f}s",
208     f" Total chunks: {num_chunks}",
209     f" Total play segments found: {len(ai_segments)}",
210     "-" * 60,

```

```

211         f" {'Chunk':<12} | {'Size':<8} | {'Upload':<8} | {'Process':<8} |
{'Gen.':<8} | {'Segments'}}",
212         "-" * 60,
213     ]
214     for i in range(num_chunks):
215         lbl = f"Chunk {i+1}/{num_chunks}"
216         m = all_metrics.get(lbl, {})
217         sz = f"{m.get('file_size_mb', 0):.1f}MB" if m else "N/A"
218         upl = f"{m.get('upload_time', 0):.1f}s" if m else "N/A"
219         prc = f"{m.get('process_time', 0):.1f}s" if m else "N/A"
220         gen = f"{m.get('gen_time', 0):.1f}s" if m else "N/A"
221         seg = m.get('segments_found', 0) if m else 0
222         summary_lines.append(f" {lbl:<12} | {sz:<8} | {upl:<8} | {prc:<8} |
{gen:<8} | {seg}")
223         summary_lines.append("-" * 60)
224         logger.info("\n".join(summary_lines))
225
226         # Clean up local temp files
227         cleanup_temp_files()
228         try:
229             os.rmdir(chunks_dir)
230         except Exception:
231             pass
232
233         logger.info(f"All chunks processed. {len(ai_segments)} total play segments
before deduplication.")
234
235     else:
236         # --- SINGLE-FILE PROCESSING (short videos) ---
237         try:
238             file_sz = os.path.getsize(video_to_upload) / (1024 * 1024)
239         except Exception:
240             file_sz = 0.0
241
242         # Get prompt from sport profile
243         prompt = sport_profile.get_prompt(chunk_duration=video_duration)
244         ai_segments, metrics = provider.analyze_video(
245             video_to_upload, prompt, chunk_duration=video_duration,
label="SingleFile"
246         )
247         summary_lines = [
248             "",
249             "-" * 50,
250             "          GEMINI SINGLE FILE SUMMARY",
251             "-" * 50,
252             f" File Size: {file_sz:.1f} MB",
253             f" Upload time: {metrics.get('upload_time', 0):.1f}s",
254             f" API Processing time: {metrics.get('process_time', 0):.1f}s",
255             f" Prompt Generation time: {metrics.get('gen_time', 0):.1f}s",
256             f" Play segments found: {len(ai_segments)}",
257             "-" * 50,
258         ]
259         logger.info("\n".join(summary_lines))
260         cleanup_temp_files()
261
262     if not ai_segments:
263         return []
264
265     # Populate SQLite DB with parsed play segments
266     segments_data = []
267     logger.info(f"Processing {len(ai_segments)} play segments through
validation...")
268
269     def parse_time(val) -> float:
270         """Converts a timestamp value to total float seconds.

```

```

271         Handles float/int pass-through, plain numeric strings, and
272         colon-separated formats (MM:SS, HH:MM:SS). Logs warnings
273         when unexpected formats are encountered."""
274         if isinstance(val, (int, float)):
275             return float(val)
276         if isinstance(val, str):
277             val = val.strip()
278             if ":" in val:
279                 # Gemini returned MM:SS or HH:MM:SS despite being asked for decimal
seconds.
280                 # Convert it, but warn so the issue is visible in logs.
281                 parts = val.split(":")
282                 parts.reverse()
283                 total = 0.0
284                 for i, p in enumerate(parts):
285                     try:
286                         total += float(p) * (60 ** i)
287                     except ValueError:
288                         pass
289                 logger.warning(f"Timestamp '{val}' was in colon-separated format
(MM:SS/HH:MM:SS) instead of decimal seconds. Converted to {total:.2f}s.")
290                 return total
291             try:
292                 return float(val)
293             except ValueError:
294                 logger.warning(f"Could not parse timestamp string '{val}' as a
number. Defaulting to 0.0.")
295                 return 0.0
296                 logger.warning(f"Unexpected timestamp type {type(val).__name__} for value
'{val}'. Defaulting to 0.0.")
297                 return 0.0
298
299         # 7. Parse timestamps and build candidate segment list
300         idx_cfg = self.config.get("indexing", {})
301         min_segment_duration = idx_cfg.get("min_segment_duration", 3.0)
302         max_segment_duration = 90.0 # Singles play segments almost never exceed this
303
304         candidates = []
305         for idx, segment in enumerate(ai_segments):
306             start = parse_time(segment.get("start_time", 0.0))
307             end = parse_time(segment.get("end_time", 0.0))
308             if start >= end:
309                 logger.info(f"Skipping segment #{idx+1}: start_time ({start:.2f}s) >=
end_time ({end:.2f}s)")
310                 continue
311                 candidates.append({
312                     "idx": idx,
313                     "start": start,
314                     "end": end,
315                     "raw": segment
316                 })
317
318         # 8. Post-hoc validation pass
319         logger.info(f"Running validation on {len(candidates)} candidate play
segments...")
320         validated = []
321         rejected = []
322
323         for c in candidates:
324             label = f"Segment #{c['idx']+1} [{c['start']:.2f}s - {c['end']:.2f}s]"
325
326             # A: Reject segments that start beyond the video
327             if video_duration > 0 and c["start"] >= video_duration:
328                 rejected.append((label, f"start_time ({c['start']:.2f}s) is beyond video
duration ({video_duration:.1f}s)"))

```

```

329         continue
330
331         # B: Clamp end_time to video duration
332         if video_duration > 0 and c["end"] > video_duration:
333             logger.info(f"{label}: end_time ({c['end']:.2f}s) exceeds video duration
334             ({video_duration:.1f}s). Clamping.")
335             c["end"] = video_duration
336
337         dur = c["end"] - c["start"]
338
339         # C: Reject segments shorter than min_segment_duration
340         if dur < min_segment_duration:
341             rejected.append((label, f"duration ({dur:.2f}s) is below minimum
342             ({min_segment_duration}s)"))
343             continue
344
345         # D: Flag suspiciously long segments
346         if dur > max_segment_duration:
347             logger.warning(
348                 f"{label}: duration ({dur:.1f}s) is unusually long
349                 (>{max_segment_duration}s). "
350                 f"This may be a detection error. Keeping but flagging."
351             )
352
353         validated.append(c)
354
355         # E: Resolve overlapping segments (keep the longer one)
356         if len(validated) > 1:
357             # Sort by start time
358             validated.sort(key=lambda r: r["start"])
359             non_overlapping = [validated[0]]
360             for curr in validated[1:]:
361                 prev = non_overlapping[-1]
362                 # Check if current segment overlaps with the previous one
363                 if curr["start"] < prev["end"]:
364                     prev_dur = prev["end"] - prev["start"]
365                     curr_dur = curr["end"] - curr["start"]
366                     kept, dropped = (prev, curr) if prev_dur >= curr_dur else (curr,
367                     prev)
368                     kept_label = f"Segment #{kept['idx']+1} [{kept['start']:.2f}s -
369                     {kept['end']:.2f}s]"
370                     dropped_label = f"Segment #{dropped['idx']+1}
371                     [{dropped['start']:.2f}s - {dropped['end']:.2f}s]"
372                     rejected.append((dropped_label, f"overlaps with {kept_label} ? kept
373                     the longer segment"))
374                     if kept == curr:
375                         non_overlapping[-1] = curr # Replace previous with current
376                     else:
377                         non_overlapping.append(curr)
378                 validated = non_overlapping
379
380         # Log validation summary
381         if rejected:
382             reject_lines = [f" ? {label}: {reason}" for label, reason in rejected]
383             logger.info(f"{len(rejected)} play segments were REJECTED:\n" +
384             "\n".join(reject_lines))
385             logger.info(f"{len(validated)} play segments passed validation (out of
386             {len(ai_segments)} from Gemini).")
387
388         stitch_cfg = self.config.get("stitching", {})
389         begin_pad = stitch_cfg.get("begin_padding", 0.5)
390         end_pad = stitch_cfg.get("end_padding", 0.5)
391
392         total_highlight_dur = 0.0
393         for c in validated:

```

```

385         total_highlight_dur += (c["end"] - c["start"]) + begin_pad + end_pad
386
387     mins = int(total_highlight_dur // 60)
388     secs = int(total_highlight_dur % 60)
389     logger.info(
390         f"Estimated final highlight video duration: {mins:02d}:{secs:02d} "
391         f"({total_highlight_dur:.1f}s total; includes {begin_pad}s begin +
{end_pad}s end padding per segment)"
392     )
393
394     # 9. Populate SQLite DB with validated play segments
395     for c in validated:
396         start = c["start"]
397         end = c["end"]
398         segment = c["raw"]
399         dur = end - start
400
401         # Extract shot_count from Gemini or compute estimate if not present
402         shot_count = segment.get("shuttle_exchange_count")
403         if shot_count is None:
404             shot_count = max(3, int(dur / 0.8)) # 1 shot every 0.8 seconds as
fallback estimate
405         else:
406             try:
407                 shot_count = int(shot_count)
408             except Exception:
409                 shot_count = max(3, int(dur / 0.8))
410
411         # Only store metrics we actually know ? no fabricated data
412         metadata = {
413             "shot_count": shot_count,
414             "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
415             "detector": model_name
416         }
417
418         segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
419
420         if segment_id:
421             segments_data.append({
422                 "id": segment_id,
423                 "start_time": start,
424                 "end_time": end,
425                 "duration": dur,
426                 "metadata": metadata
427             })
428
429     return segments_data
430
431     # Register the segmenter
432     segmenter_registry.register(GeminiPlaySegmenter)
433

```

11. assistant (2026-06-06T15:20:58.531Z)

I have all the files read. Let me verify a couple of cross-module contracts to be accurate about edges ? specifically the DB methods and the provider/runner interfaces.

12. user (2026-06-06T15:21:00.112Z)

```

116:     def add_play_segment(self, video_id: str, start_time: float, end_time: float, server:
Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]] = None)
-> Optional[int]:
130:     def add_event(self, segment_id: int, timestamp: float, event_type: str, player:

```

```
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
143:     def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
166:     def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
```

13. user (2026-06-06T15:21:01.564Z)

```
30:class WasbConfig:
42:     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
57:class WasbRunner:
68:     def healthcheck(self) -> tuple:
105:     def run_predict(self, video_win_path: str, output_win_dir: str) -> Optional[str]:
```

14. assistant (2026-06-06T15:21:04.605Z)

I have enough to map the subsystem accurately. The provider's `analyze\_video` and `sport\_profile.get\_prompt` signatures are clear from usage. Let me return the structured map.

15. user (2026-06-06T15:22:45.431Z)

Structured output provided successfully

16. assistant (2026-06-06T15:22:57.666Z)

I've mapped the Segmenters (indexer) subsystem. The structured output is complete, covering all 7 files with roles, key symbols, cross-module data contracts, dataflow edges, and gotchas.

Key takeaways for a future engineer:

- base.py defines the ABC + a singleton `segmenter\_registry`; concrete segmenters self-register at import time, and `\_\_init\_\_.py` is the de-facto registration manifest.
- The three "hybrid" segmenters share an identical 4-phase shape (candidate windows -> AI handoff via ThreadPoolExecutor -> DB + candidate linking). tracknet_hybrid.py and wasb_hybrid.py are near-verbatim copy-paste twins (only the WSL2 runner and a `queue\_length` field differ) - any fix must be applied to both.
- gemini.py is the pure-AI path (no local Phase 2) with heavy post-hoc validation; motion.py is a legacy CV baseline that fabricates all metadata/events with `random` and is the only segmenter using `add\_event`/server/receiver.
- Notable footguns: `\_PERSON\_CLASS\_ID=1` off-by-one vs old ultralytics; WSL2 healthcheck gates the trajectory hybrids; `debug\_duration` silently truncates in gemini; gemini hard-codes `max\_workers=5` ignoring `ai\_max\_workers`.